

Hardware Accelerator for Object Identification

Project Abstract

by

CMPE 295A

Gokul Harikrishnan (gokul.harikrishnan@sjsu.edu)

Alec Cheng (alec.cheng@sjsu.edu)

Yen-Che Cheng (yen-che.cheng@sjsu.edu)

Project Advisor

Professor Bhawandeep Singh Harsh

September 2026

ABSTRACT

Hardware Accelerator for Object Identification

By

Gokul Harikrishnan, Alec Cheng, Yen-che Cheng

Modern computer vision relies heavily on deep neural networks such as convolutional neural networks and vision transformers for real-time object detection and tracking. These models are increasingly being deployed directly on edge systems, including autonomous vehicles, robotics, drones, and smart cameras, where local inference can reduce latency, improve reliability, and reduce dependence on continuous network connectivity.

Deploying object-detection networks on resource-constrained FPGA platforms presents several challenges. Neural-network inference requires large numbers of matrix operations as well as frequent movement of weights and intermediate feature data. Smaller Zynq-based platforms have limited DSP resources, on-chip memory, and external-memory bandwidth compared with high-performance GPUs. Quantization can reduce these requirements, but using the same numerical precision across an entire network may be inefficient because different layers can tolerate reduced precision differently. This creates a tradeoff between detection accuracy, execution speed, memory usage, and FPGA resource utilization.

This project will investigate a mixed-precision hardware accelerator for object detection on a Zynq-based FPGA platform. Different numerical precisions will be evaluated to determine how they affect detection accuracy and hardware resource requirements. A configurable accelerator will then be developed to accelerate computationally intensive portions of the network while reducing programmable-logic usage. The completed system will be compared with a software implementation using metrics such as detection accuracy, latency, memory bandwidth, and FPGA resource utilization.